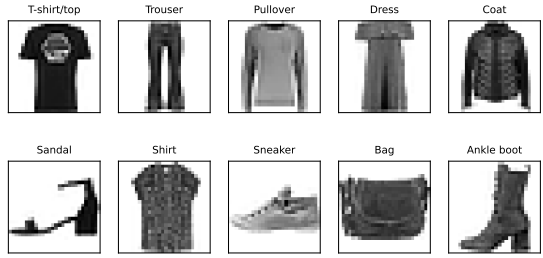
How do you *look* at 784-dimensional data?

A photo of a shirt is $28 \times 28 = \mathbf{784}$ numbers; gene expression $\sim$20,000; a text embedding $\sim$768. You cannot plot that.

Dimensionality reduction: replace many features with a *few* new ones that keep most of the structure.

Bridge from clustering — both are **unsupervised** (no y): clustering groups the **rows** (samples), DR compresses the **columns** (features).



Running dataset: Fashion-MNIST, 12,000 garment photos (28×28 grayscale, 10 classes). Each image = 784 pixel values; we will squeeze those 784 down to 2 to *see* the data.

Why reduce dimensions?

- **Visualization** — see high-dimensional data in 2-D.
- **Preprocessing / compression** — faster models, less storage, fewer features before clustering or a classifier.
- **Denoising** — drop tiny-variance directions that are *often* just noise.

And a deeper reason, next: in high dimensions, distance itself stops working.

The curse of dimensionality

Predict-first: in 100 dimensions, how much closer is your *nearest* point than your *farthest*?

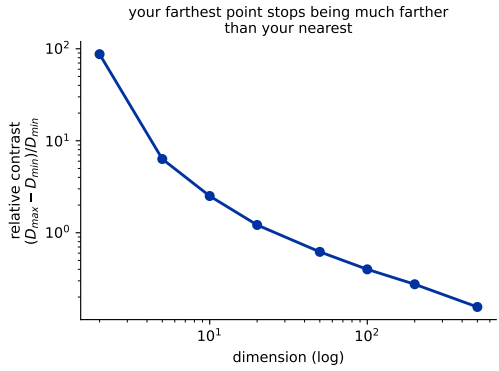
The curse of dimensionality

Predict-first: in 100 dimensions, how much closer is your *nearest* point than your *farthest*?

Almost the same. In 100 dimensions the farthest point is only about **40% farther** than the nearest; in 2 dimensions it is roughly **87 times** farther. Distances **concentrate**, so distance-based methods (k-means, kNN, DBSCAN…) degrade.

Reduce first, and distances mean something again.

Full treatment in our homework: `math/30_curse_of_dimensionality.qmd`.



Relative contrast $(D_{max} - D_{min})/D_{min}$ from random query points (Beyer et al. 1999).

Outline

PCA: the idea and the math

PCA in practice

Nonlinear DR for visualization: t-SNE & UMAP

Choosing a method

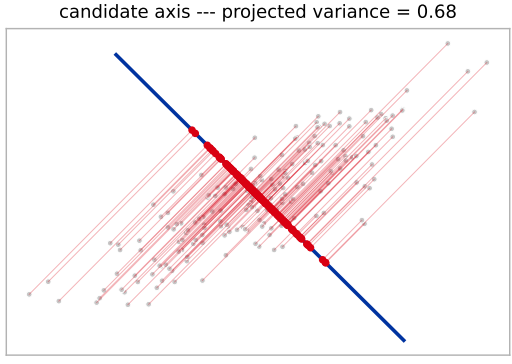
Connections and wrap-up

Principal Component Analysis

The linear workhorse: new axes along the directions of greatest variance.

PCA: keep the directions of greatest spread

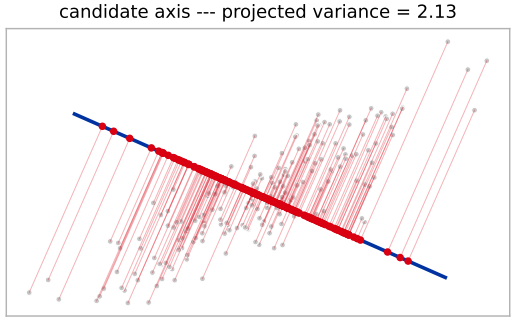
Sweep a direction through the cloud and **project** the points onto it (red dots). The **projected variance $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$** — how spread out those red dots are — rises and falls, peaking at the **principal axis**. Keep the **top-k** such axes (here 2-D→1-D; for the 784-D garments we will keep 2).



PCA does not search or iterate — the eigenvectors (next frames) give these axes directly; the sweep just shows what "maximum variance" means.

PCA: keep the directions of greatest spread

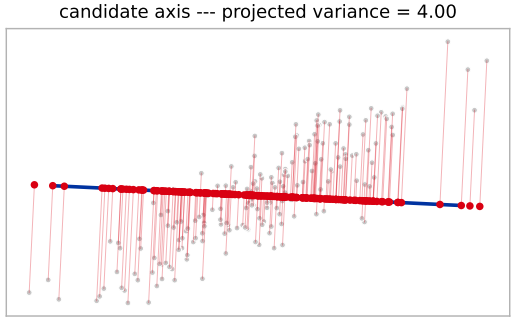
Sweep a direction through the cloud and **project** the points onto it (red dots). The **projected variance $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$** — how spread out those red dots are — rises and falls, peaking at the **principal axis**. Keep the **top-k** such axes (here 2-D→1-D; for the 784-D garments we will keep 2).



PCA does not search or iterate — the eigenvectors (next frames) give these axes directly; the sweep just shows what "maximum variance" means.

PCA: keep the directions of greatest spread

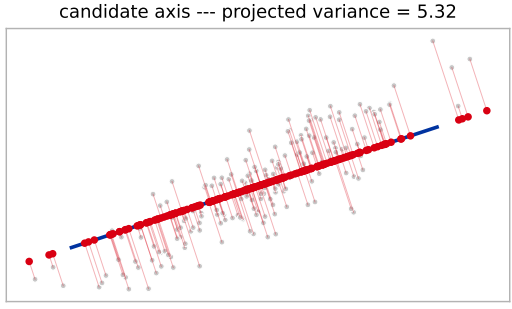
Sweep a direction through the cloud and **project** the points onto it (red dots). The **projected variance $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$** — how spread out those red dots are — rises and falls, peaking at the **principal axis**. Keep the **top-k** such axes (here 2-D→1-D; for the 784-D garments we will keep 2).



PCA does not search or iterate — the eigenvectors (next frames) give these axes directly; the sweep just shows what "maximum variance" means.

PCA: keep the directions of greatest spread

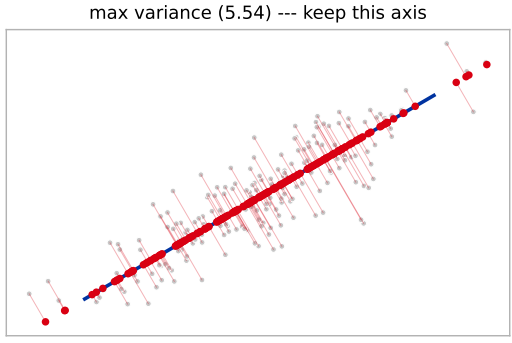
Sweep a direction through the cloud and **project** the points onto it (red dots). The **projected variance $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$** — how spread out those red dots are — rises and falls, peaking at the **principal axis**. Keep the **top-k** such axes (here 2-D→1-D; for the 784-D garments we will keep 2).



PCA does not search or iterate — the eigenvectors (next frames) give these axes directly; the sweep just shows what "maximum variance" means.

PCA: keep the directions of greatest spread

Sweep a direction through the cloud and **project** the points onto it (red dots). The **projected variance $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$** — how spread out those red dots are — rises and falls, peaking at the **principal axis**. Keep the **top-k** such axes (here 2-D→1-D; for the 784-D garments we will keep 2).



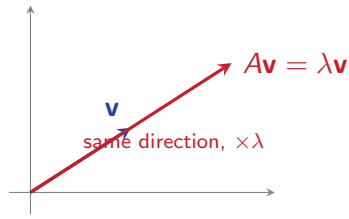
PCA does not search or iterate — the eigenvectors (next frames) give these axes directly; the sweep just shows what "maximum variance" means.

Refresher: eigenvectors and eigenvalues

An **eigenvector v** of a matrix A keeps its *direction* when A acts on it; it only gets scaled by the **eigenvalue** λ:

$$A\mathbf{v} = \lambda\mathbf{v}.$$

A **symmetric** matrix — like a covariance — has real eigenvalues and **orthogonal** eigenvectors. *That is why the principal axes come out perpendicular.*



Derivation (1): maximize variance

Step 0 — center the data (subtract each feature's mean). Without it, $X^\top X$ measures spread about the *origin*, not about the data.

Variance of the data projected onto a unit direction $\mathbf{w}$ is $\mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w}$ ($\boldsymbol{\Sigma}$ = covariance of the centered data). Maximize it subject to $\|\mathbf{w}\| = 1$. Form the **Lagrangian** and set its gradient to zero:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^\top \boldsymbol{\Sigma} \mathbf{w} - \lambda(\mathbf{w}^\top \mathbf{w} - 1), \qquad \nabla_{\mathbf{w}} \mathcal{L} = 2\boldsymbol{\Sigma}\mathbf{w} - 2\lambda \mathbf{w} = 0$$

$$\implies \quad \boldsymbol{\Sigma}\mathbf{w} = \lambda\, \mathbf{w}.$$

The principal directions are the **eigenvectors of the covariance matrix**; each eigenvalue λ is the **variance captured** along that direction.

Derivation (2): explained variance

- Eigenvalues sorted **descending** $\Rightarrow$ principal components come out **ordered by variance**; take the top k eigenvectors.
- **Explained-variance ratio** of component i is $\lambda_i / \sum_j \lambda_j$ (first PC of the garments $\approx 29\%$; first two $\approx 47\%$).
- The scores are **uncorrelated**: $\mathrm{Cov}(Z) = W^\top \boldsymbol{\Sigma} W = \mathrm{diag}(\lambda_1, \ldots, \lambda_k)$. This is the precise sense in which PCA *decorrelates* — not that it "removes correlated features".

Two views, one answer. Maximizing variance **is** minimizing **reconstruction error**: projecting onto the top-k axes and lifting back loses the least. That is the basis for compression and denoising later.

The SVD connection

In practice PCA is computed by the **singular value decomposition** of the centered data:

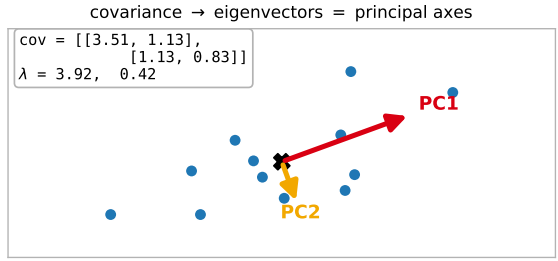
$$X_{\text{centered}} = USV^\top, \qquad \text{columns of } V = \text{principal components.}$$

- More numerically stable than forming the covariance matrix explicitly.
- **TruncatedSVD** (a.k.a. *LSA* for text) skips centering, so it works on **sparse** data — centering would destroy the sparsity.

PCA by hand (2-D)

1. Center the points (subtract the mean, the **X**).
2. Form the 2×2 **covariance** matrix.
3. Its **eigenvectors** are the principal axes; **eigenvalues** λ are the variances (arrow length $\propto \sqrt{\lambda}$).

PC1 captures most of the spread; PC2 is perpendicular and small.

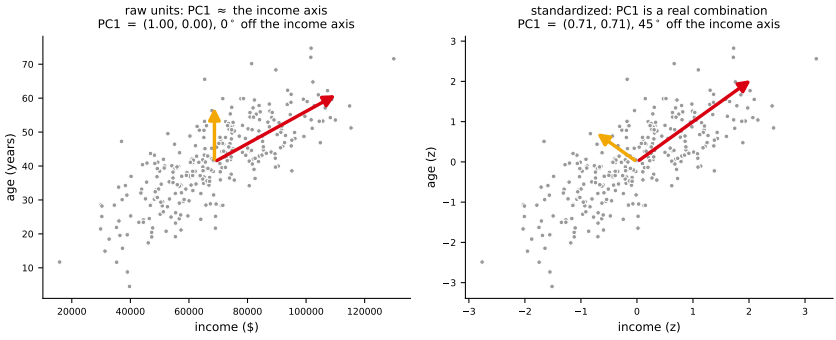


PCA in practice

Scaling, how many components to keep, what they mean, and where PCA bites.

Centering vs scaling

- PCA **always centers** (scikit-learn does it for you) — mandatory.
- PCA does **not standardize** — that is your choice.



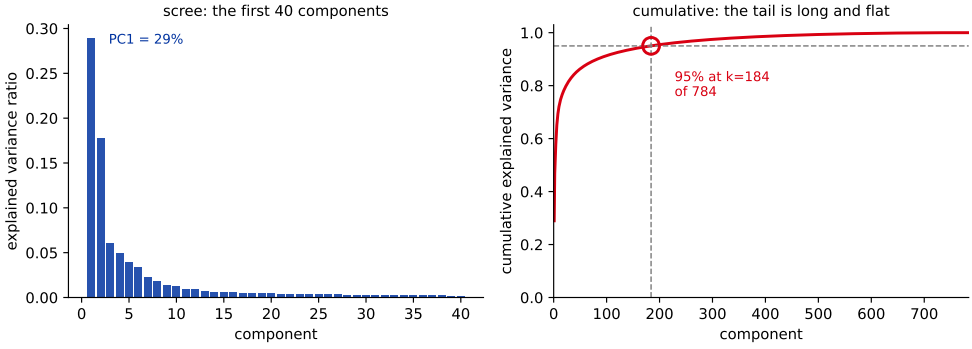
Trap: variance is scale-dependent. In dollars, PC1 is (1.00, 0.00) — literally the income axis, and age contributes nothing. Standardized, PC1 is (0.71, 0.71): the real shared direction. **Standardize** unless your features already share units (like pixel intensities, which is why we do not standardize the garments).

How many components to keep?

Predict-first: the garments are 784-D and the first PC alone is 29%. How many components do you need to reach **95%**?

How many components to keep?

Predict-first: the garments are 784-D and the first PC alone is 29%. How many components do you need to reach **95%**?



184 — almost a quarter of them, not a handful. **Scree plot** (left): variance per component, and it collapses almost immediately. **Cumulative** (right): the tail is long and flat, and that long tail is where the detail lives.

When PCA feeds a model, do not pick k by a variance threshold at all — tune it by cross-validation on the *downstream* metric.

Reading a biplot

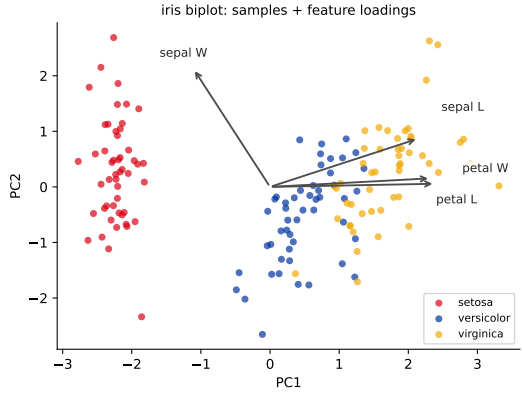
A **biplot** shows two things on the same PC1–PC2 axes:

- **dots** = the samples projected onto PC1/PC2 (colored by species here);
- **arrows** = the original features (their *loadings*).

How to read an arrow:

- **direction** = the way that feature increases;
- **length** = how strongly it drives these two PCs;
- arrows pointing the **same way** = correlated features.

So petal length & width point together (correlated) and separate the species along PC1; sepal width points elsewhere. *Components are combinations — only partly interpretable.*



Reconstruction: compression and denoising

Project to k scores, then **lift back** to the original space:

$$\mathbf{z} = W_k^\top(\mathbf{x} - \bar{\mathbf{x}}), \qquad \hat{\mathbf{x}} = \bar{\mathbf{x}} + W_k\,\mathbf{z}.$$

"Lift back" = the **mean plus a weighted sum of the kept axes**:

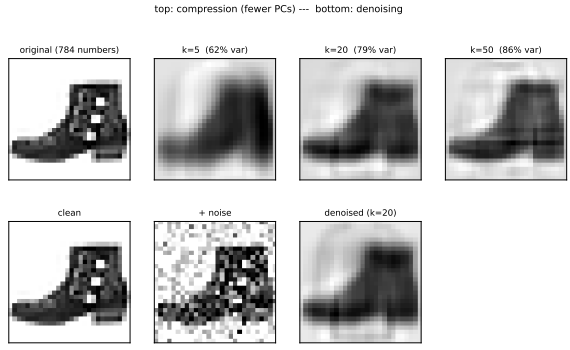
$$\hat{\mathbf{x}} = \bar{\mathbf{x}} + \textstyle\sum_{i=1}^{k} z_i\,\mathbf{v}_i$$

($\mathbf{v}_i$ the principal directions, z_i the scores). Fewer k = more compression (blurrier).

784 numbers $\rightarrow$ **50** and the garment is still clearly itself.

Callback: the blurriness at k=5 is exactly the variance the scree plot said we threw away. The squared reconstruction error is $\sum_{i>k}\lambda_i$ (the dropped eigenvalues); as a *fraction* of the total, that is 1− cumulative EVR.

Noise lives in the small-variance directions you dropped, so the rebuild is also **denoised**.



PCA pitfalls

- **Not feature selection** — components are *combinations* of all features, not a chosen subset. DR is feature **extraction** (build new features); the other family, feature **selection**, keeps a subset of the originals (drop low-variance or correlated columns, L1, . . .). Extraction gives you better axes; selection gives you back your original, readable ones.
- **Linear only** — it cannot unfold a curved manifold (next section).
- **Sensitive** to outliers and to feature scale.
- Components can be **hard to interpret**.

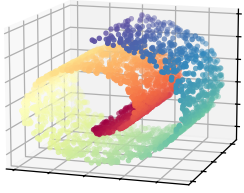
Nonlinear methods

When the structure is a curved manifold, linear PCA is not enough — t-SNE and UMAP preserve *local* neighborhoods.

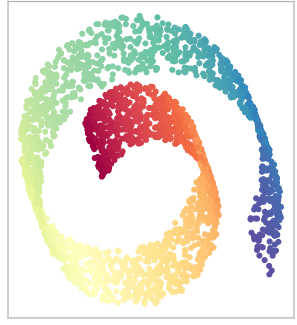
Why nonlinear?

PCA is **linear**: it can rotate and project, but it cannot *unfold* a curved manifold. Nonlinear methods instead preserve **local neighborhoods** — who is near whom.

the data: a 2-D sheet
rolled up in 3-D



PCA (linear): a shadow ---
the layers stay overlapped



UMAP: one sweep of colour,
end to end, nothing overlapping

Color = position along the roll. **PCA** gives you a *shadow*: the roll stays rolled, so points on *adjacent turns* sit side by side even though they are far apart along the sheet. **UMAP** sweeps the colors once, end to end. It did not lay the sheet out straight — it had no reason to. Topology preserved, geometry not.

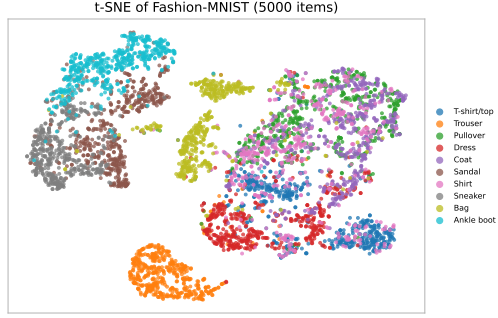
t-SNE

Turn distances into **similarity probabilities**, high-D and low-D, then match them.

- High-D: $p_{ij} = \frac{1}{2n}(p_{j|i} + p_{i|j})$ (Gaussian neighborhoods).
- Low-D: q_{ij} from a **Student-t** kernel (heavy tail = fixes the "crowding" problem).
- Minimize $\mathrm{KL}(P \,\|\, Q)$ by gradient descent (iterative, *stochastic*).

Perplexity $\approx$ effective #neighbors (typ. 5–50).

Sandal, sneaker and ankle boot are each other's two nearest classes here — and so are pullover, coat and shirt. **t-SNE recovered the semantics from raw pixels**, with no labels.



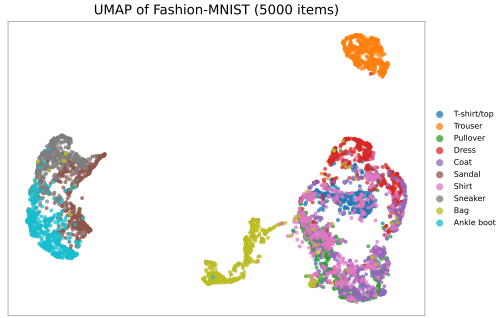
van der Maaten & Hinton (2008).

UMAP

Build a **fuzzy nearest-neighbor graph**, then find a low-D layout whose graph matches it (minimize a graph **cross-entropy**).

- `n_neighbors`: small = local detail, large = global shape.
- `min_dist`: how tightly points may pack.
- **Faster** than t-SNE, scales better, keeps *more* global structure.

This frame is *what UMAP gives you*. **How it works** — the fuzzy graph, ρ_i and σ_i, the cross-entropy — is the whole of the next lecture (L13c).



McInnes et al. (2018).

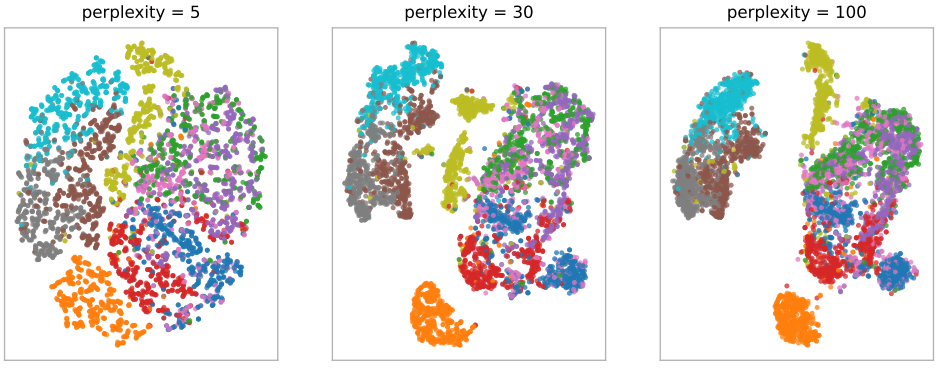
Caveats: t-SNE / UMAP can mislead

Predict-first: same data, same algorithm, one hyperparameter moved. Will the picture be the same?

Caveats: t-SNE / UMAP can mislead

Predict-first: same data, same algorithm, one hyperparameter moved. Will the picture be the same?

same data, three perplexities --- the picture changes

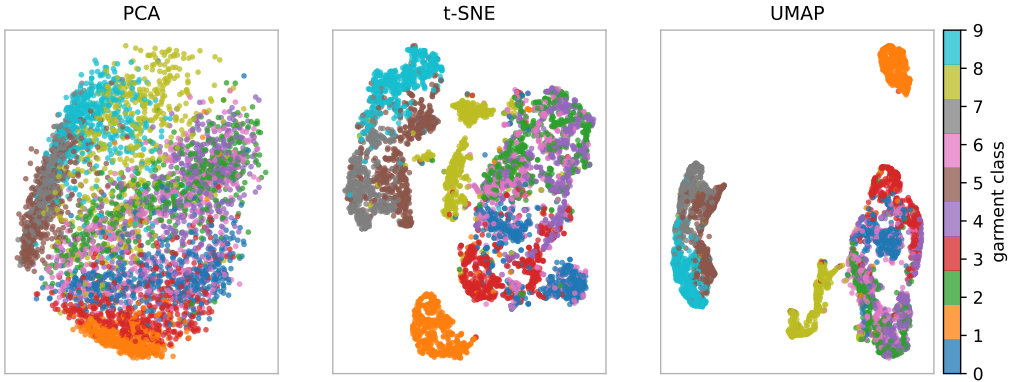


So: **cluster sizes and between-cluster distances are not meaningful**; results are **stochastic**; **do not feed the 2-D embedding to a downstream model** — it is for the eyes. (Wattenberg et al., distill.pub 2016.)

Which one?

PCA, t-SNE, UMAP — side by side, then a decision table.

PCA vs t-SNE vs UMAP on the garments



PCA (linear) piles the classes on top of each other; t-SNE and UMAP pull them apart. PCA is for preprocessing; t-SNE / UMAP are for *looking*.

In practice the nonlinear methods here were run on the top 50 PCs, not the raw 784 pixels: PCA first as a denoiser and accelerator, then t-SNE / UMAP for the picture. That stack is the normal recipe, not a shortcut.

Decision table

	Linear?	Global?	Determin.?	Use it for
PCA	yes	yes	yes	preprocessing, compression, denoising
t-SNE	no	no	no	visualizing local cluster structure
UMAP	no	some	no (seeded)	visualization, faster / larger data

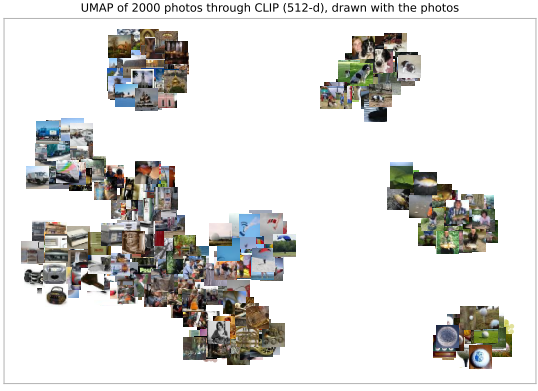
Rule of thumb: reach for **PCA** first (fast, cheap, reversible); use **t-SNE / UMAP** when you specifically want a 2-D *picture* of the structure.

Connections

Where DR meets the rest of the course.

What this is actually used for

The garments were a teaching dataset. The real job: 2000 photos through **CLIP** into **512-D** vectors, then UMAP to 2-D, each point drawn as its own photo.



Nobody labelled this, and there is no legend because you do not need one. Similar *meanings* landed together — the same picture you will build for retrieval in ch17 (RAG).

A wider view (one line each)

- **Kernel PCA** — PCA in a kernel feature space: nonlinear components without leaving the PCA framework.
- **Autoencoders** — a neural network that learns a low-D "bottleneck": nonlinear, learned DR. *(Neural-networks chapter.)*
- **DR as preprocessing** — standardize $\rightarrow$ PCA $\rightarrow$ then cluster or train; this is the cluster-after-PCA pipeline from the previous deck, and it denoises too.

When NOT to reduce

DR throws information away — so do not reduce reflexively:

- PCA is **unsupervised**: it ranks directions by variance *with no knowledge of* y. The direction that separates your labels can sit in a **low-variance** component PCA discards (a thin signal axis vs. a fat nuisance axis).
- **Tree models** (forests, boosting) usually do not need it.
- Always try the model *without* DR first; add it only if it helps.

Recap

- **Why:** visualize, compress, denoise — and beat the curse of dimensionality (in 100-D your farthest point is only ∼40% farther than your nearest).
- **PCA** — center, then take the top eigenvectors of the covariance (via SVD); the scores come out uncorrelated and ordered by variance; linear, fast, reversible. Great for preprocessing. On the garments, 95% of the variance needs **184 of 784** components — but 50 already rebuilds a recognizable shirt.
- **t-SNE / UMAP** — nonlinear, preserve local neighborhoods, excellent for 2-D *pictures* — but sizes/distances are not meaningful and they are for the eyes only.
- Reach for PCA first; don't reduce reflexively (it is unsupervised, and it loses information).

Next (L13c): UMAP, opened up — the fuzzy neighbor graph, why each point gets its own distance scale, and what the cross-entropy is really doing. After that, unsupervised learning is closed and we move to a new family of models.